

Linux Application Development - Day 4

Process Management and IPC

Instructor: [Your Name]

Duration: 8 hours (including labs)

Welcome to Day 4!

Previous Days Recap

- ✓ **Day 1:** Build tools, GCC, libraries, Make
- ✓ **Day 2:** Debugging (GDB, Valgrind), syscalls, memory
- ✓ **Day 3:** File systems, file I/O, mmap, locking

Today's Goal

-  **Processes** - fork(), exec(), wait(), lifecycle
-  **IPC** - Pipes, shared memory, message queues, semaphores
-  **Signals** - Async event handling, signal safety

Day 4 Agenda

Time	Topic
09:00-10:30	Process Lifecycle and Creation
10:30-10:45	Break
10:45-12:00	IPC: Pipes, Shared Memory, Semaphores
12:00-13:00	Lunch
13:00-14:30	Signals and Signal Handling
14:30-14:45	Break
14:45-16:00	Advanced IPC and Daemons
16:00-17:00	Hands-On Lab

Section 1: Process Fundamentals

What is a Process?

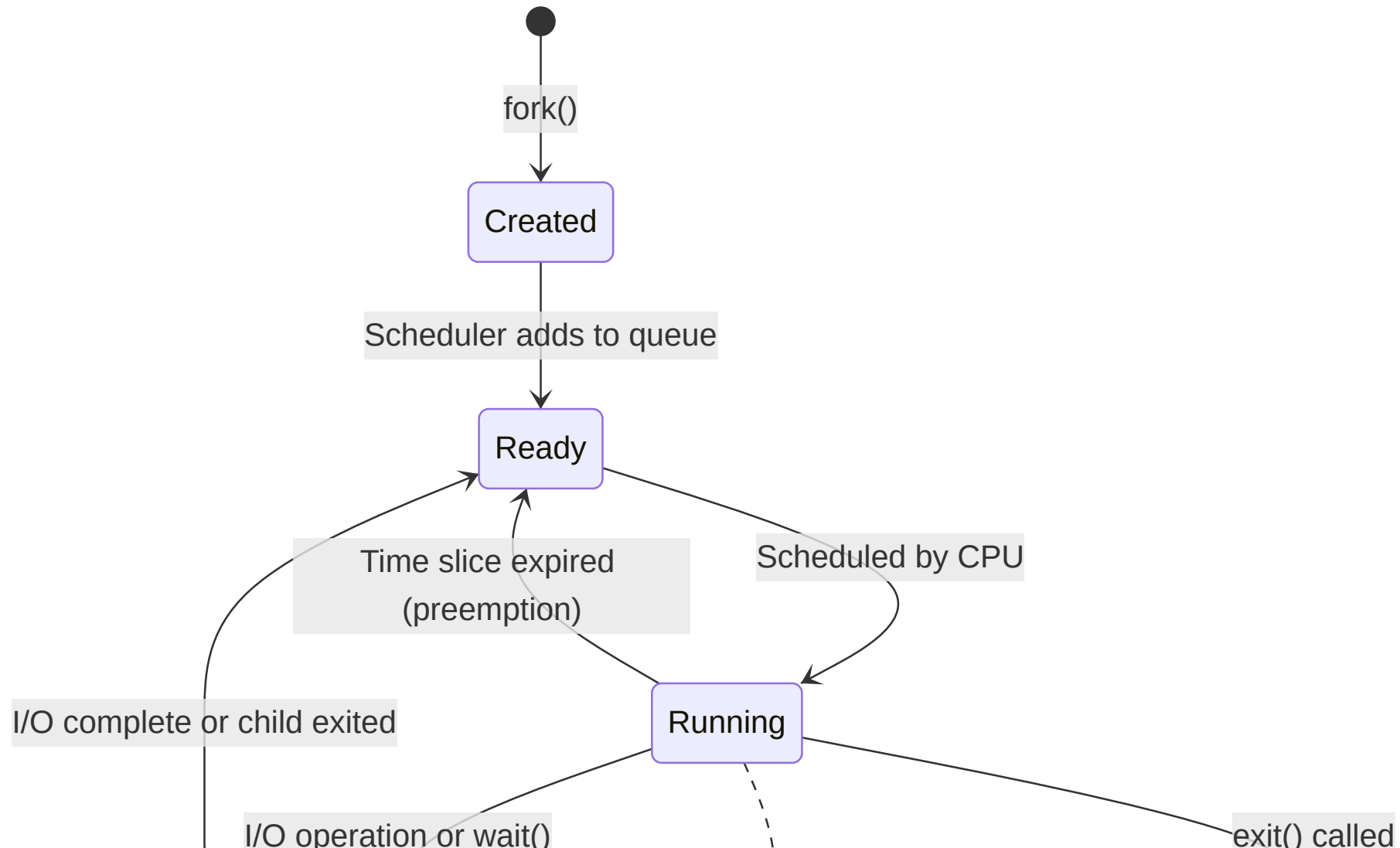
Program vs Process

- **Program** - Executable file on disk (static)
- **Process** - Running instance of a program (dynamic)

Process Components

- **Code (text segment)** - Executable instructions
- **Data** - Global/static variables
- **Heap** - Dynamic memory (malloc)
- **Stack** - Function calls, local variables
- **File descriptors** - Open files
- **Process ID (PID)** - Unique identifier

Process States



Process Information

```
# View all processes
```

```
ps aux
```

```
ps -ef
```

```
# Process tree
```

```
pstree
```

```
ps axjf
```

```
# Detailed process info
```

```
ls -l /proc/$$      # Current shell
```

```
# Process specific info
```

```
cat /proc/$$/status
```

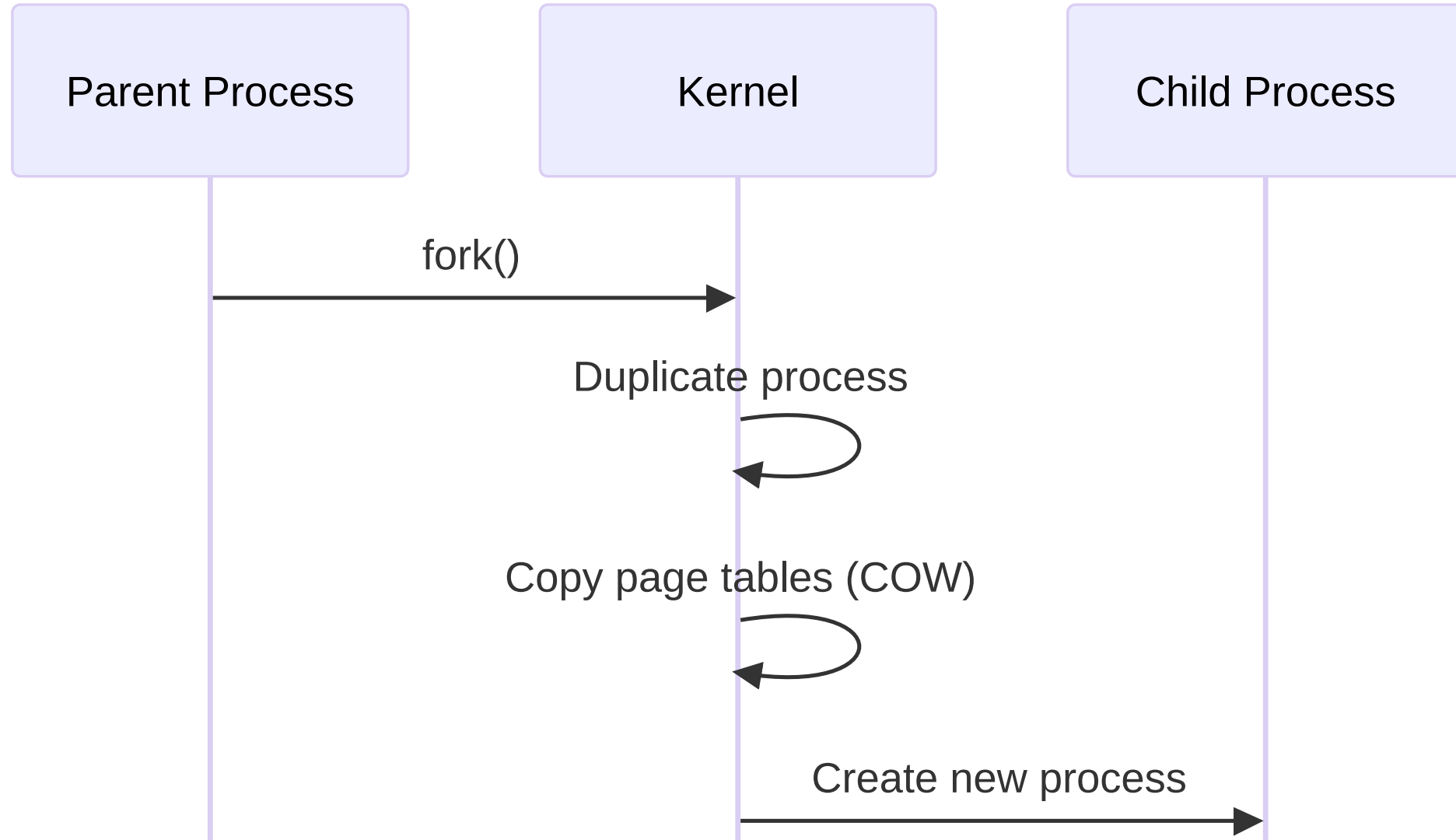
```
cat /proc/$$/cmdline
```

```
cat /proc/$$/environ
```

```
ls -l /proc/$$/fd/      # Open file descriptors
```

```
cat /proc/$$/maps       # Memory mappings
```

Creating Processes: fork()



fork() Basics

```
#include <unistd.h>
#include <sys/types.h>

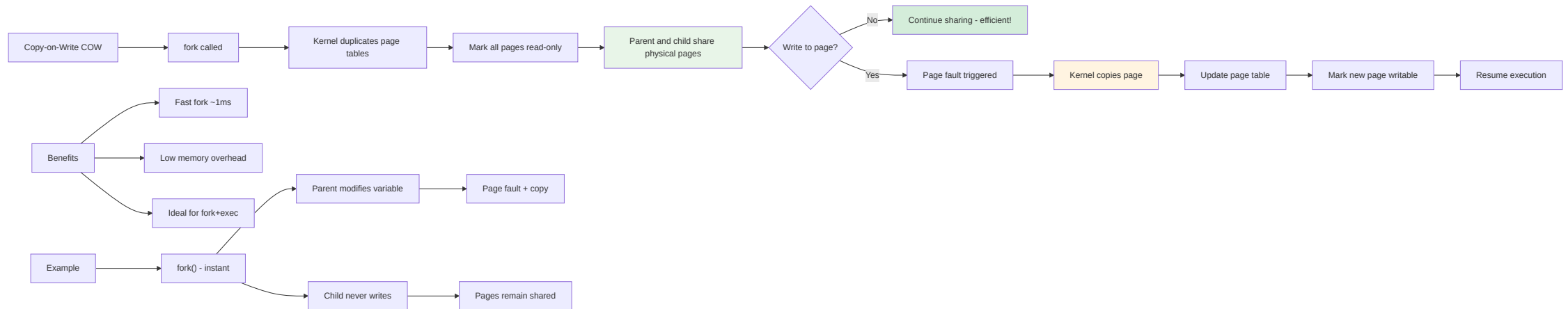
pid_t fork(void);

// Returns:
// - In parent: child's PID (> 0)
// - In child: 0
// - On error: -1

pid_t pid = fork();

if (pid == -1) {
    perror("fork");
    return -1;
} else if (pid == 0) {
    // Child process code
    printf("I'm the child, PID=%d, PPID=%d\n", getpid(), getppid());
    // ... child work ...
    exit(0);
} else {
    // Parent process code
    printf("I'm the parent, PID=%d, child is %d\n", getpid(), pid);
    // ... parent work ...
    wait(NULL); // Wait for child
}
```

Copy-on-Write (COW)



fork() Behavior

```
int global_var = 100;

int main() {
    int local_var = 200;

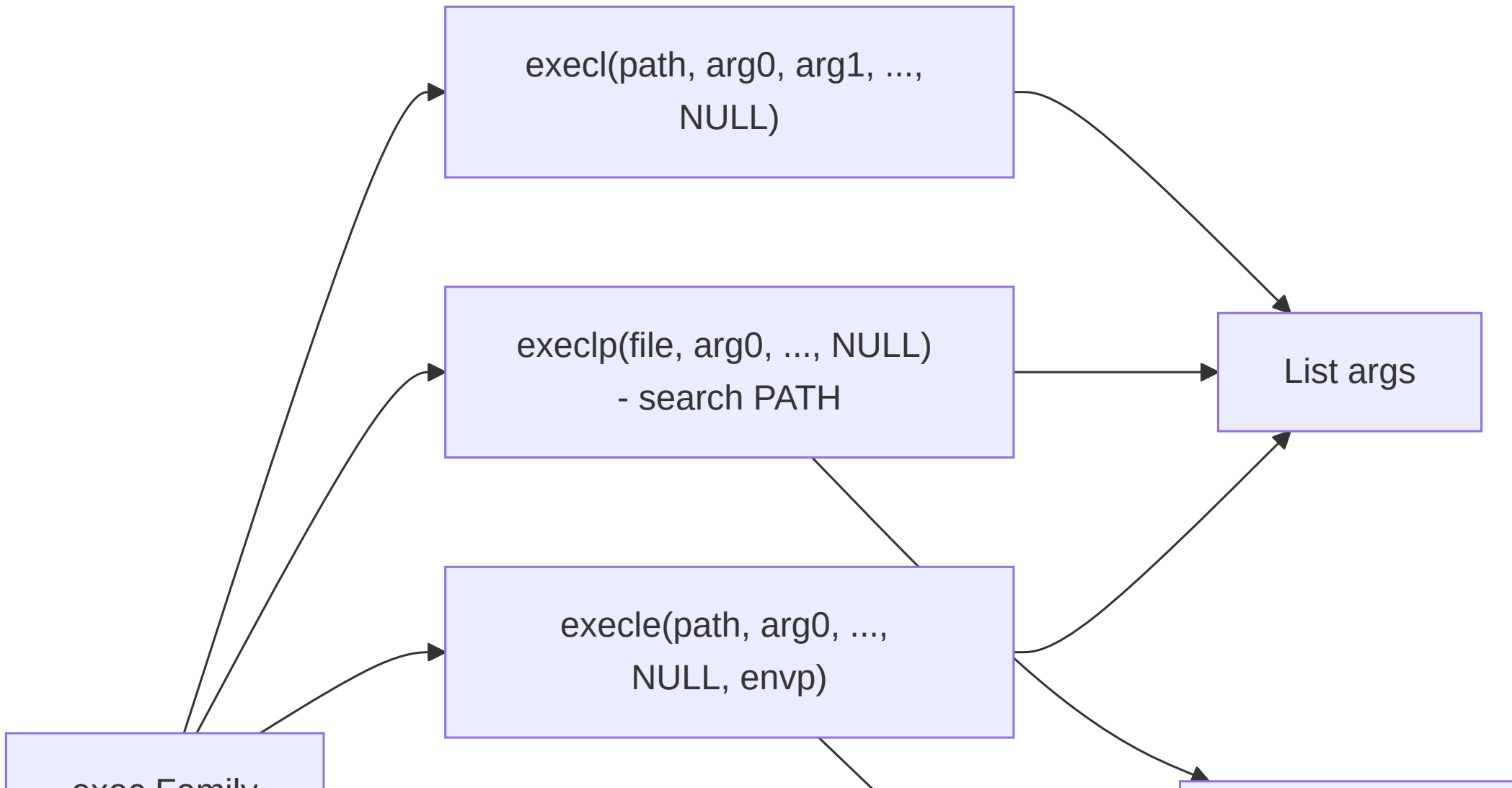
    printf("Before fork: global=%d, local=%d\n", global_var, local_var);

    pid_t pid = fork();

    if (pid == 0) {
        // Child modifies variables
        global_var = 999;
        local_var = 888;
        printf("Child: global=%d, local=%d\n", global_var, local_var);
        exit(0);
    } else {
        wait(NULL);
        // Parent's variables unchanged!
        printf("Parent: global=%d, local=%d\n", global_var, local_var);
    }
}
```

```
// Output:
// Before fork: global=100, local=200
// Child: global=999, local=888
// Parent: global=100, local=200
```

exec() Family



Using exec()

```
#include <unistd.h>

// exec replaces current process image
// execl: list of arguments
execl("/bin/ls", "ls", "-l", "/tmp", NULL);

// execlp: search PATH
execlp("ls", "ls", "-l", NULL);

// execv: vector (array) of arguments
char *args[] = {"ls", "-l", "/tmp", NULL};
execv("/bin/ls", args);

// execve: with environment (actual syscall)
char *envp[] = {"PATH=/bin:/usr/bin", "USER=me", NULL};
execve("/bin/ls", args, envp);

// If exec succeeds, code below never executes!
perror("exec");
exit(1);
```

fork() + exec() Pattern

```
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        // Child: execute new program
        execl("/bin/ls", "ls", "-l", NULL);
        // Only reached if exec fails
        perror("exec");
        _exit(1); // Use _exit() in child
    }

    // Parent: wait for child
    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Child exited with status %d\n", WEXITSTATUS(status));
    }

    return 0;
}
```

Waiting for Child Processes

```
#include <sys/wait.h>

// Wait for any child
pid_t pid = wait(&status);

// Wait for specific child
waitpid(child_pid, &status, 0);

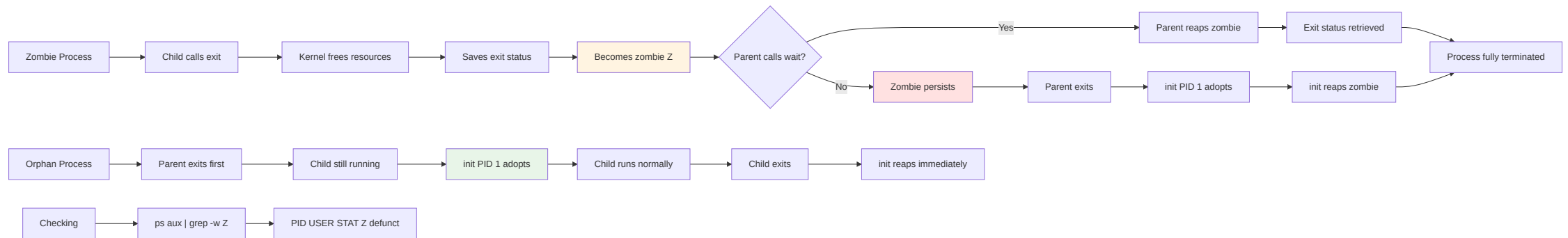
// Non-blocking wait
waitpid(-1, &status, WNOHANG);

// Check exit status
if (WIFEXITED(status)) {
    int exit_code = WEXITSTATUS(status);
    printf("Exited normally with %d\n", exit_code);
}

if (WIFSIGNALED(status)) {
    int signal = WTERMSIG(status);
    printf("Terminated by signal %d\n", signal);
}

if (WIFSTOPPED(status)) {
    int signal = WSTOPSIG(status);
    printf("Stopped by signal %d\n", signal);
}
```

Zombie and Orphan Processes



Preventing Zombies

```
#include <signal.h>

// Method 1: Explicit wait
pid_t pid = fork();
if (pid > 0) {
    wait(NULL); // Block until child exits
}

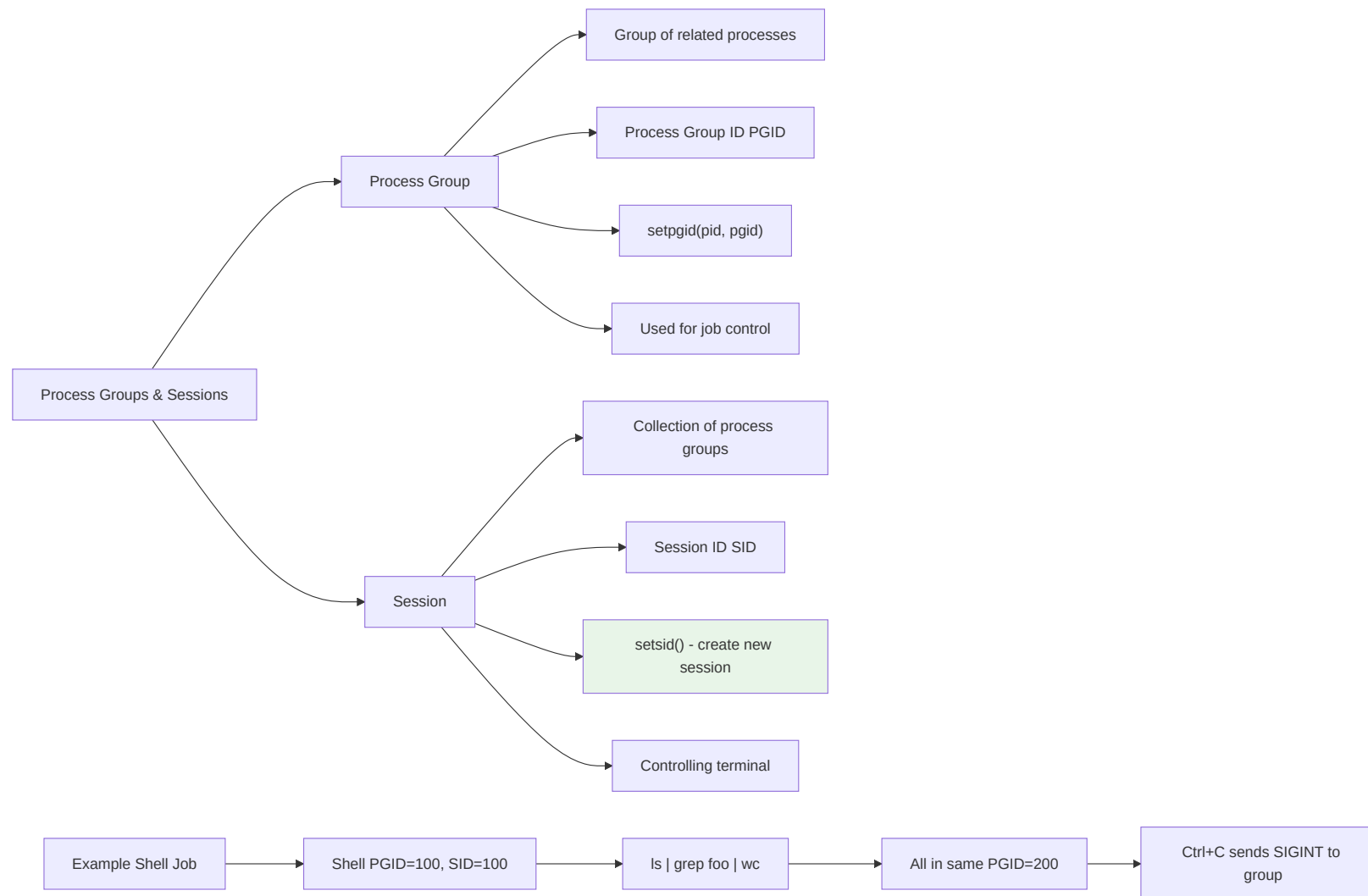
// Method 2: SIGCHLD handler
void sigchld_handler(int sig) {
    // Reap all zombie children
    while (waitpid(-1, NULL, WNOHANG) > 0);
}

int main() {
    struct sigaction sa;
    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;
    sigaction(SIGCHLD, &sa, NULL);

    // Fork children...
}

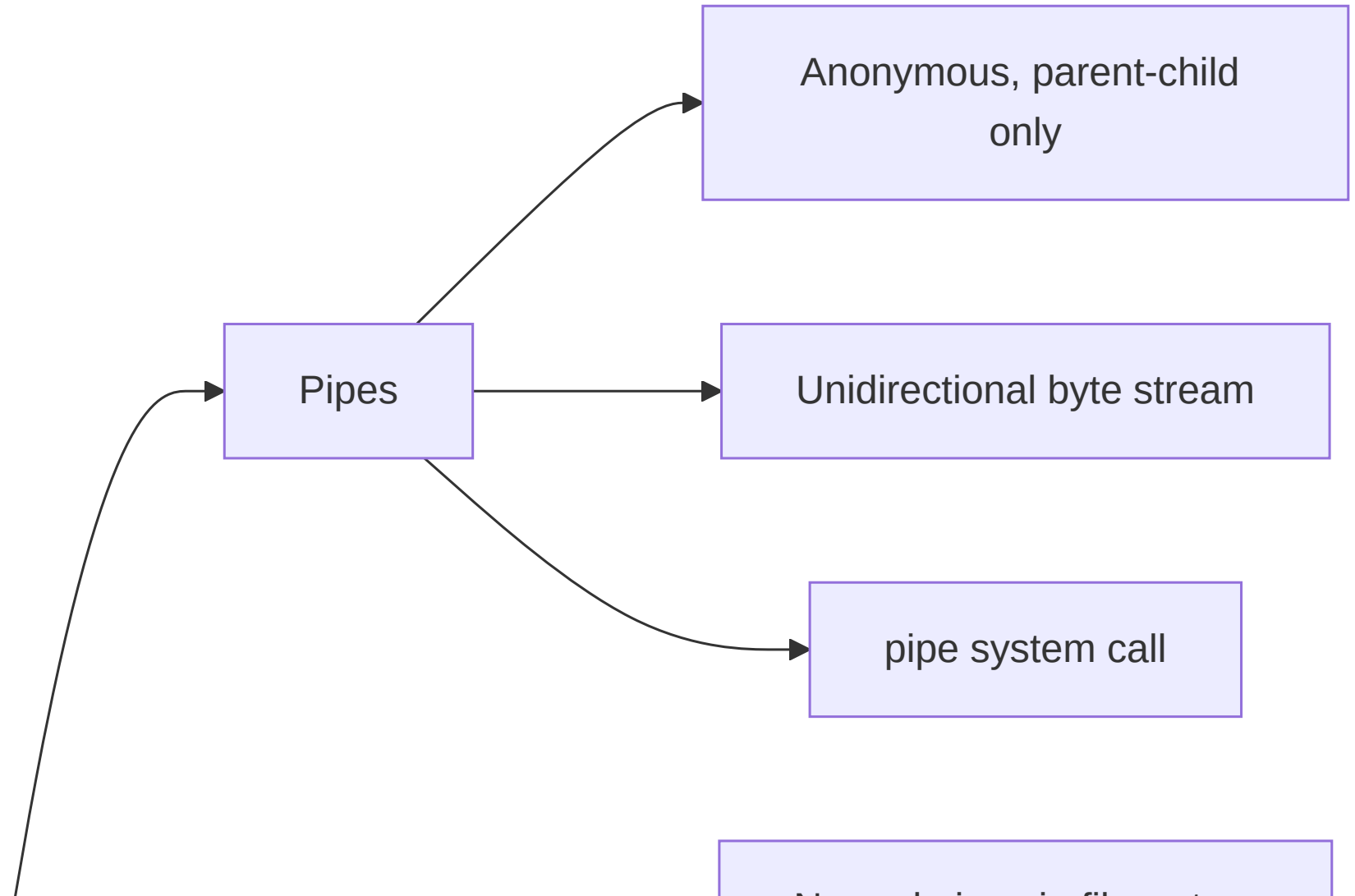
// Method 3: Ignore SIGCHLD (automatic reaping)
signal(SIGCHLD, SIG_IGN);
```

Process Groups and Sessions

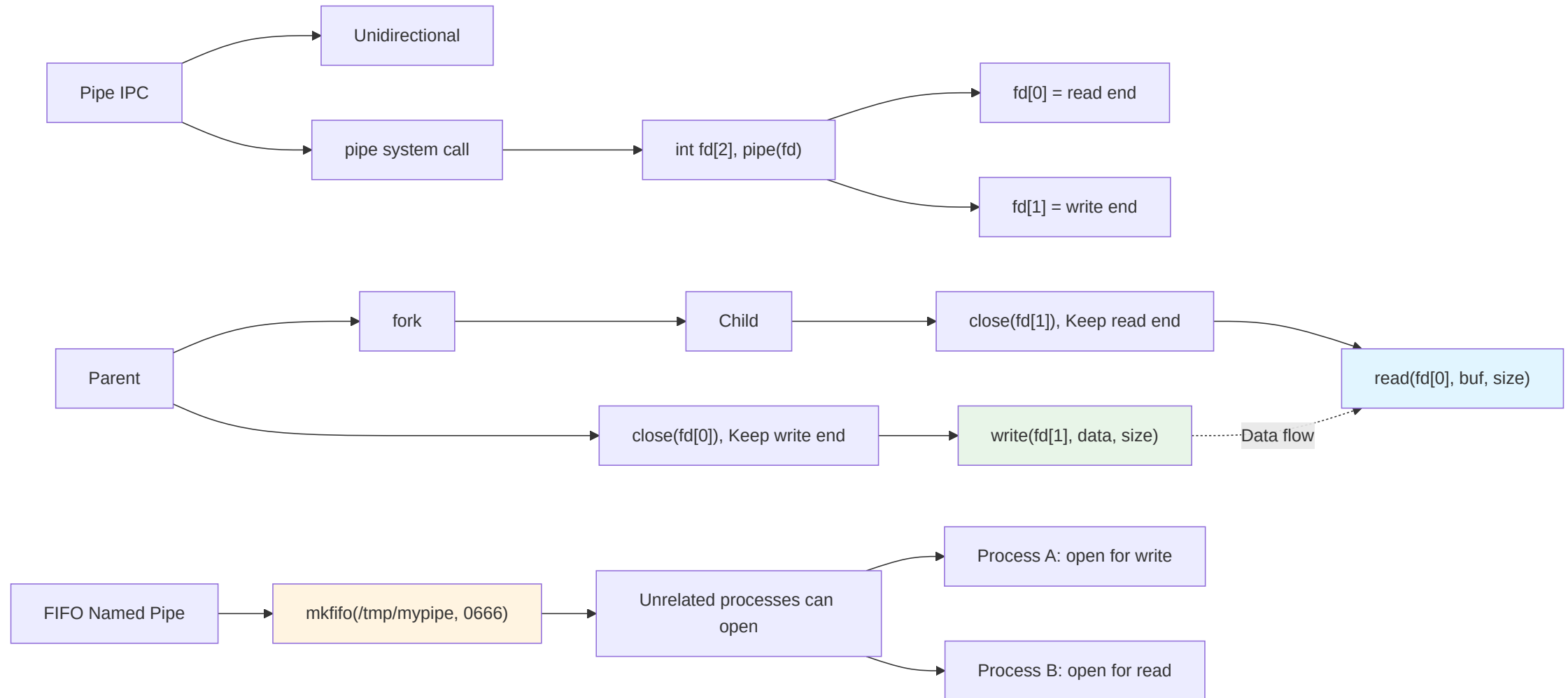


Section 2: Inter-Process Communication (IPC)

IPC Mechanisms Overview



Pipes: Anonymous Communication



Using pipe()

```
#include <unistd.h>

int main() {
    int pipefd[2];
    char buf[100];

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();

    if (pid == 0) {
        // Child: read from pipe
        close(pipefd[1]); // Close write end

        ssize_t n = read(pipefd[0], buf, sizeof(buf));
        write(STDOUT_FILENO, buf, n);

        close(pipefd[0]);
        exit(0);
    } else {
        // Parent: write to pipe
        close(pipefd[0]); // Close read end

        const char *msg = "Hello from parent!\n";
        write(pipefd[1], msg, strlen(msg));

        close(pipefd[1]);
        wait(NULL);
    }
}
```

Creating Command Pipelines

```
// Implement: ls | grep .c | wc -l

int pipe1[2], pipe2[2];
pipe(pipe1);
pipe(pipe2);

if (fork() == 0) {
    // First child: ls
    dup2(pipe1[1], STDOUT_FILENO);
    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);
    execlp("ls", "ls", NULL);
}

if (fork() == 0) {
    // Second child: grep
    dup2(pipe1[0], STDIN_FILENO);
    dup2(pipe2[1], STDOUT_FILENO);
    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);
    execlp("grep", "grep", ".c", NULL);
}

if (fork() == 0) {
    // Third child: wc
    dup2(pipe2[0], STDIN_FILENO);
    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);
    execlp("wc", "wc", "-l", NULL);
}

// Parent closes all pipe ends and waits
close(pipe1[0]); close(pipe1[1]);
close(pipe2[0]); close(pipe2[1]);
while (wait(NULL) > 0);
```

FIFOs (Named Pipes)

```
#include <sys/stat.h>
#include <fcntl.h>

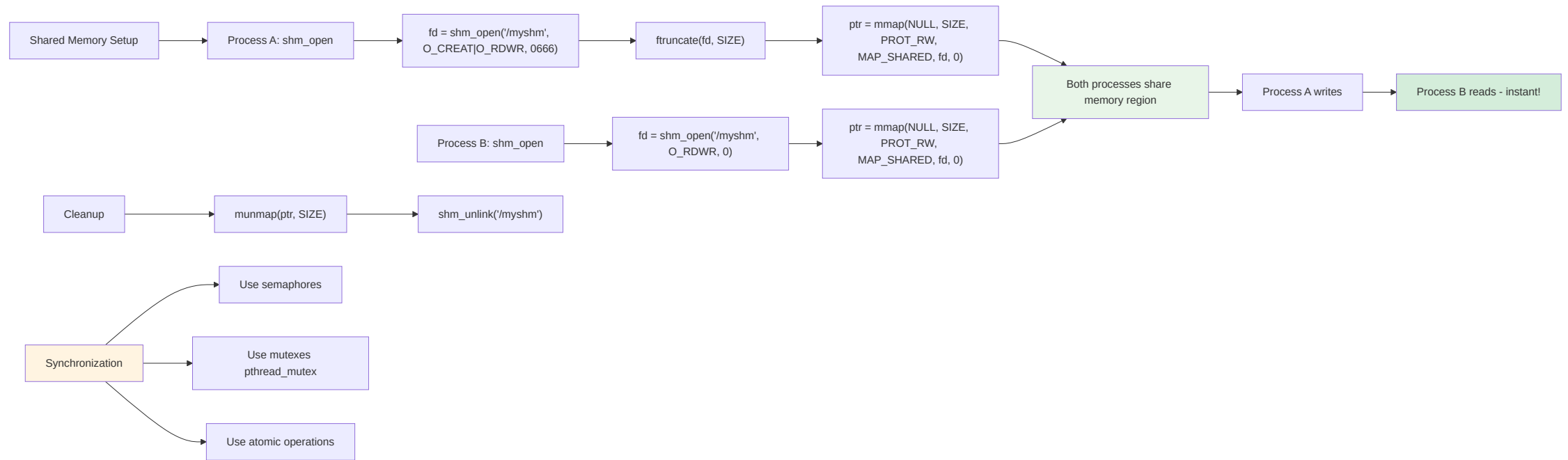
// Create FIFO
mkfifo("/tmp/myfifo", 0666);

// Process A: Writer
int fd = open("/tmp/myfifo", O_WRONLY);
write(fd, "Hello", 5);
close(fd);

// Process B: Reader (can be unrelated process!)
int fd = open("/tmp/myfifo", O_RDONLY);
char buf[10];
read(fd, buf, 5);
printf("Received: %s\n", buf);
close(fd);

// Cleanup
unlink("/tmp/myfifo");
```


Shared Memory IPC



POSIX Shared Memory

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>

// Process A: Create and write
int fd = shm_open("/myshm", O_CREAT | O_RDWR, 0666);
ftruncate(fd, 4096);

void *ptr = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                 MAP_SHARED, fd, 0);

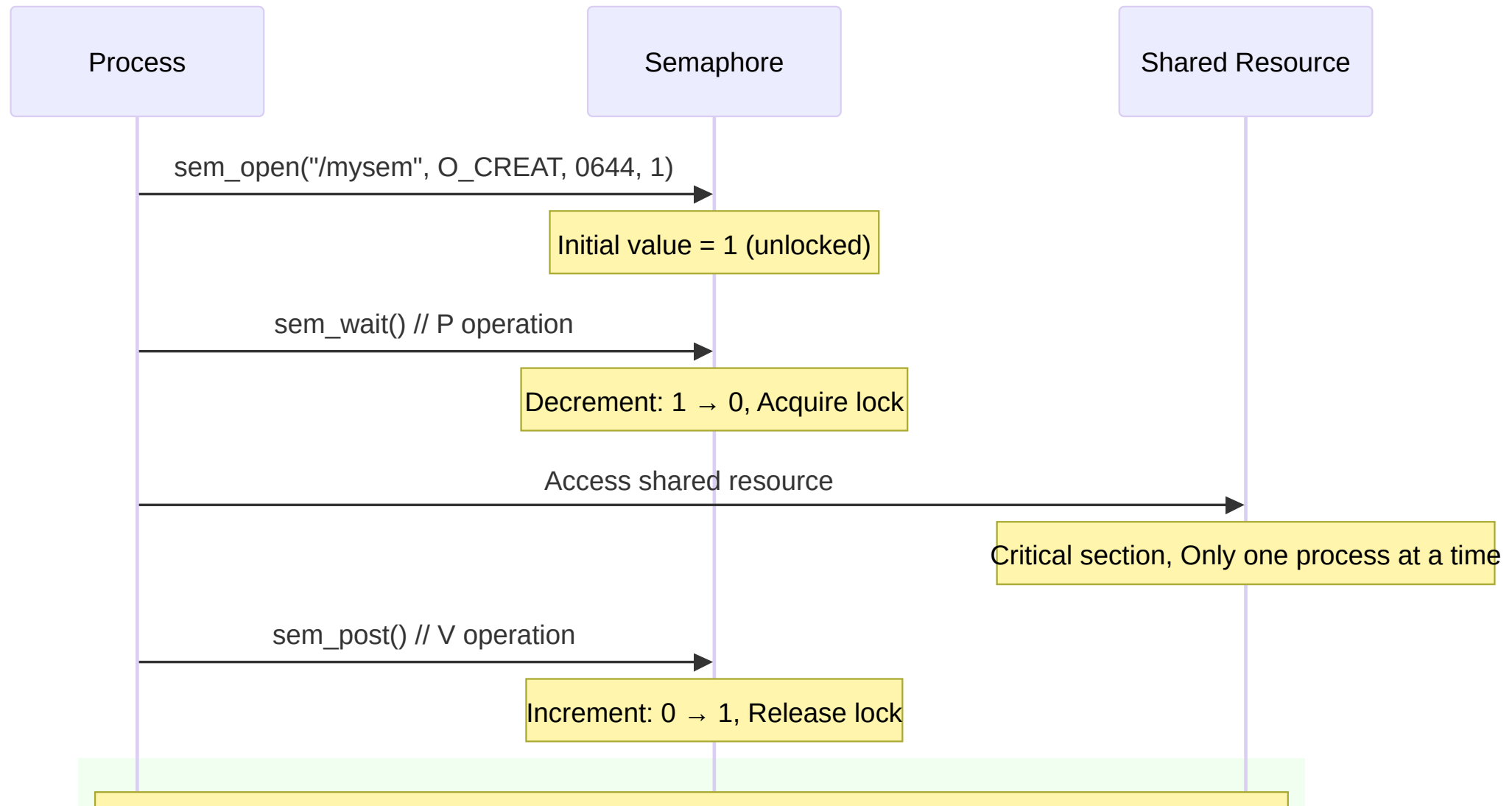
strcpy((char *)ptr, "Shared data!");

// Process B: Read
int fd = shm_open("/myshm", O_RDONLY, 0);
void *ptr = mmap(NULL, 4096, PROT_READ, MAP_SHARED, fd, 0);

printf("Data: %s\n", (char *)ptr);

// Cleanup
munmap(ptr, 4096);
shm_unlink("/myshm");
```

Semaphores for Synchronization



POSIX Semaphores

```
#include <semaphore.h>
#include <fcntl.h>

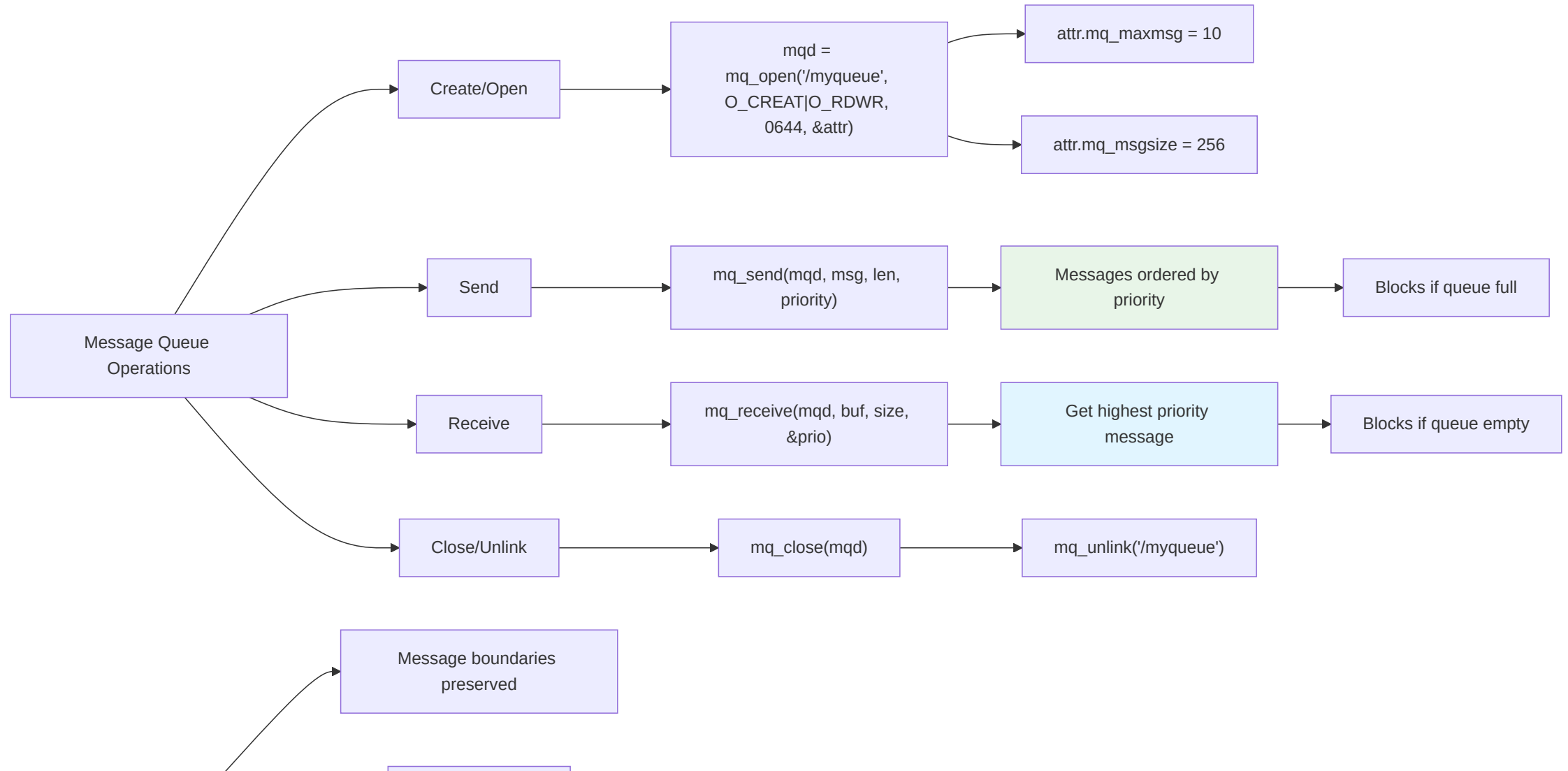
// Create/open named semaphore
sem_t *sem = sem_open("/mysem", O_CREAT, 0644, 1);
// Initial value = 1 (unlocked)

// Process A: Acquire lock
sem_wait(sem); // Decrement (P operation)
// Critical section
sem_post(sem); // Increment (V operation)

// Process B: Same code
sem_wait(sem); // Blocks if A holds lock
// Critical section
sem_post(sem);

// Cleanup
sem_close(sem);
sem_unlink("/mysem");
```

Message Queues



POSIX Message Queues

```
#include <mqueue.h>

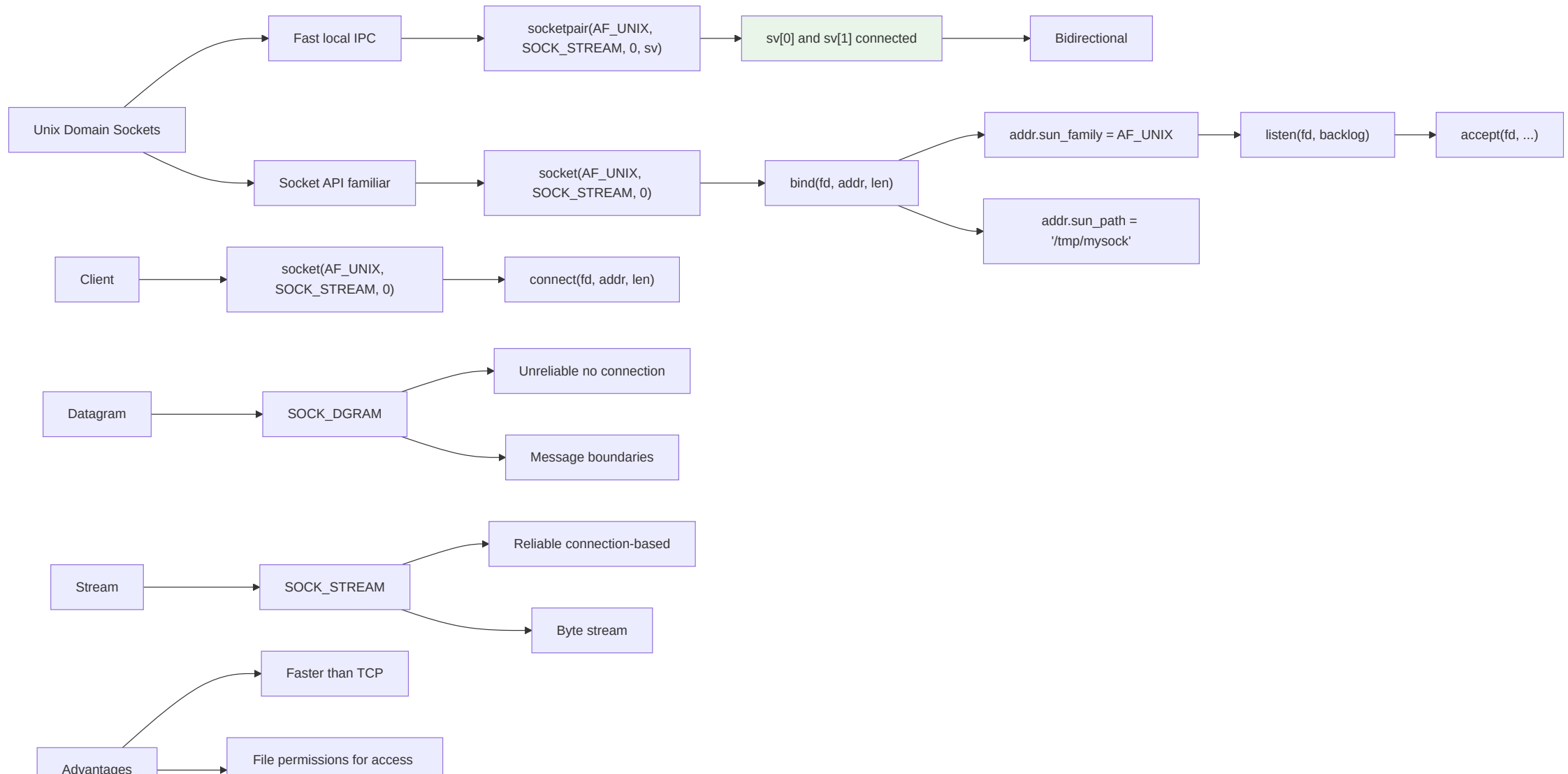
struct mq_attr attr;
attr.mq_maxmsg = 10;
attr.mq_msgsize = 256;

// Sender
mqd_t mq = mq_open("/mymq", O_CREAT | O_WRONLY, 0644, &attr);
const char *msg = "Hello, queue!";
mq_send(mq, msg, strlen(msg) + 1, 0); // Priority 0
mq_close(mq);

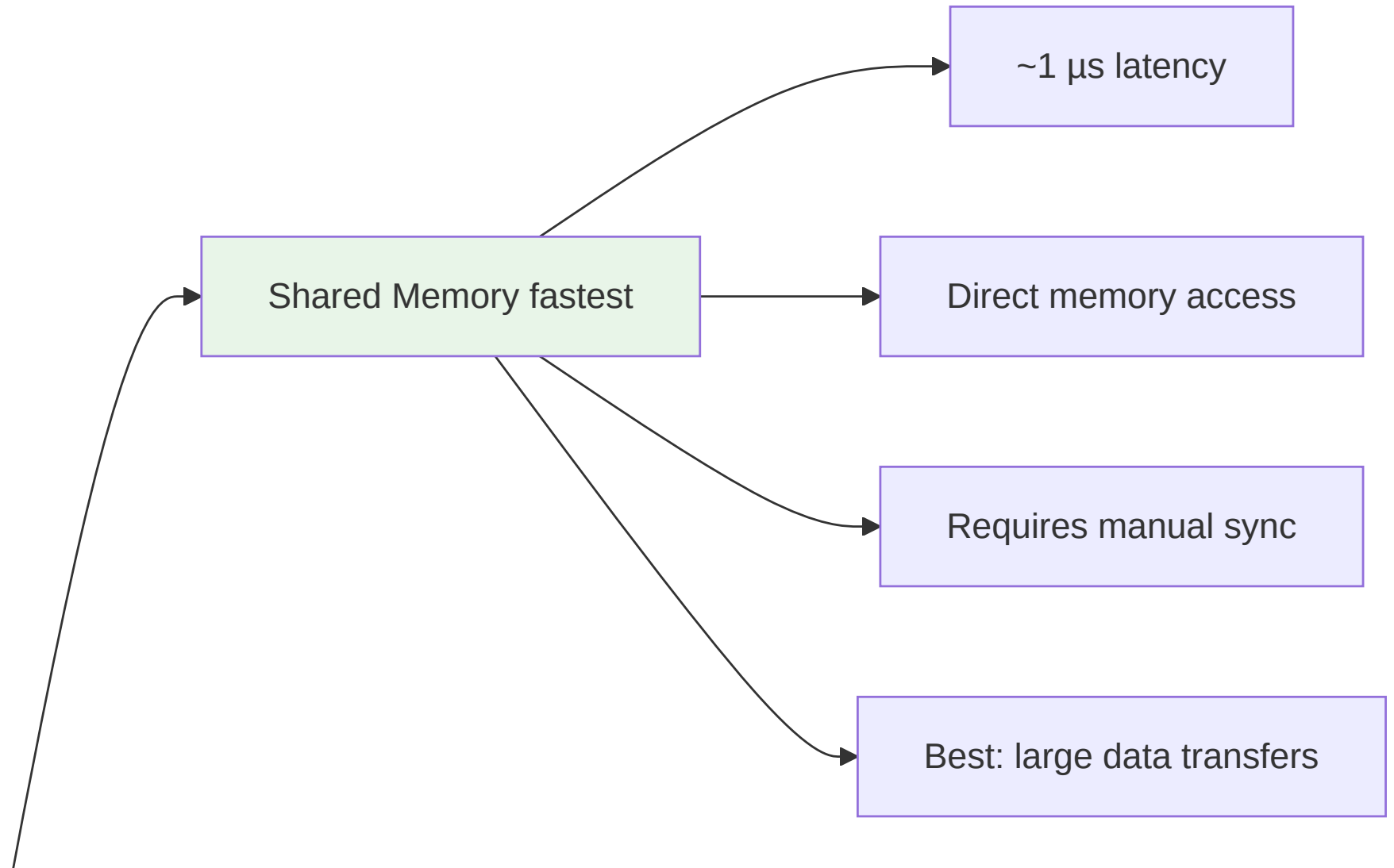
// Receiver
mqd_t mq = mq_open("/mymq", O_RDONLY);
char buffer[256];
unsigned int prio;
ssize_t n = mq_receive(mq, buffer, 256, &prio);
printf("Received (%u): %s\n", prio, buffer);
mq_close(mq);

// Cleanup
mq_unlink("/mymq");
```

Unix Domain Sockets



IPC Performance Comparison



Section 3: Signals

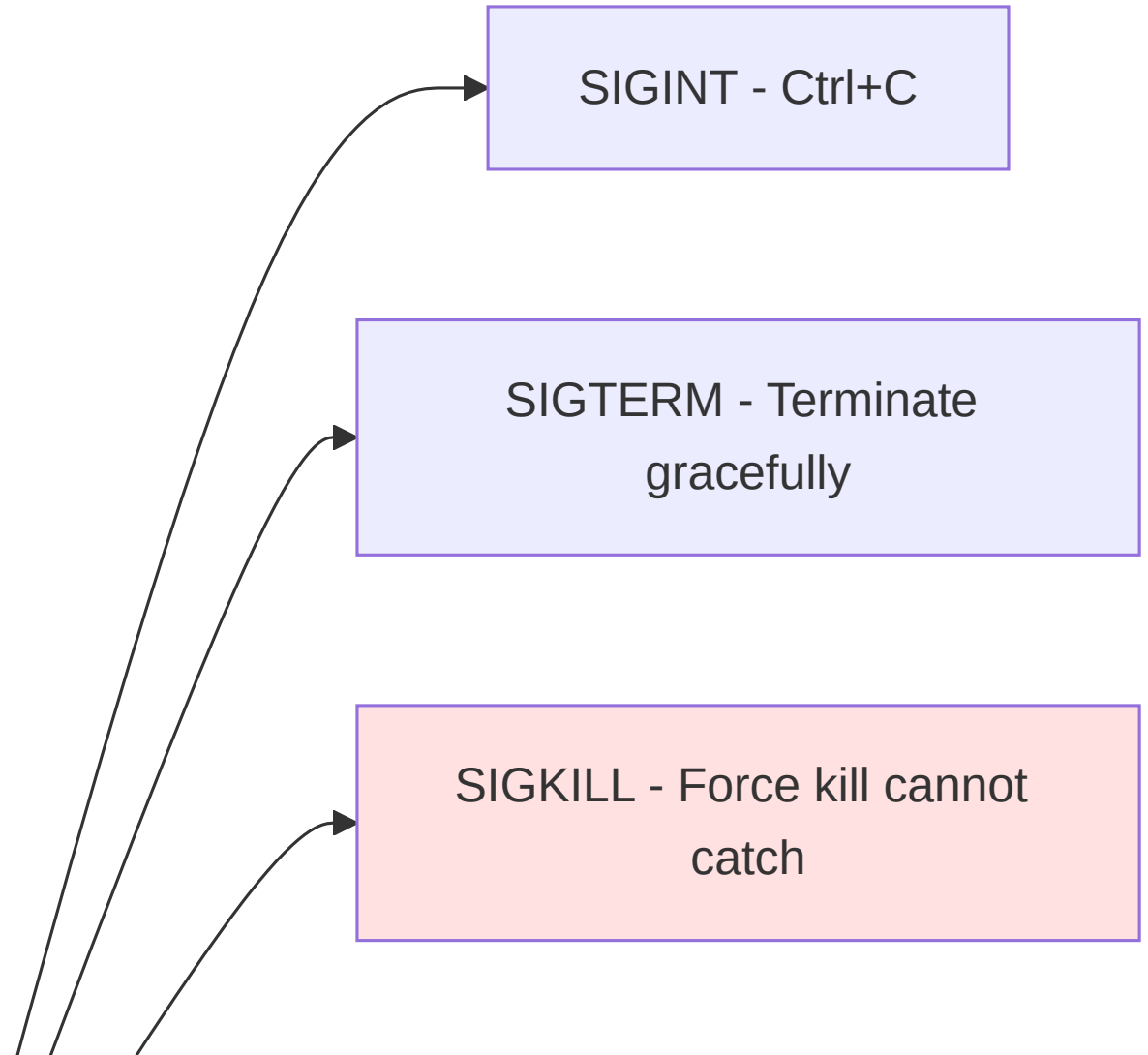
What are Signals?

Software Interrupts

- **Asynchronous notifications** to processes
- **Generated by** - kernel, other processes, self
- **Delivered** - at kernel's convenience
- **Default actions** - terminate, ignore, stop, continue

Signals enable event-driven programming

Signal Types



Sending Signals

```
#include <signal.h>

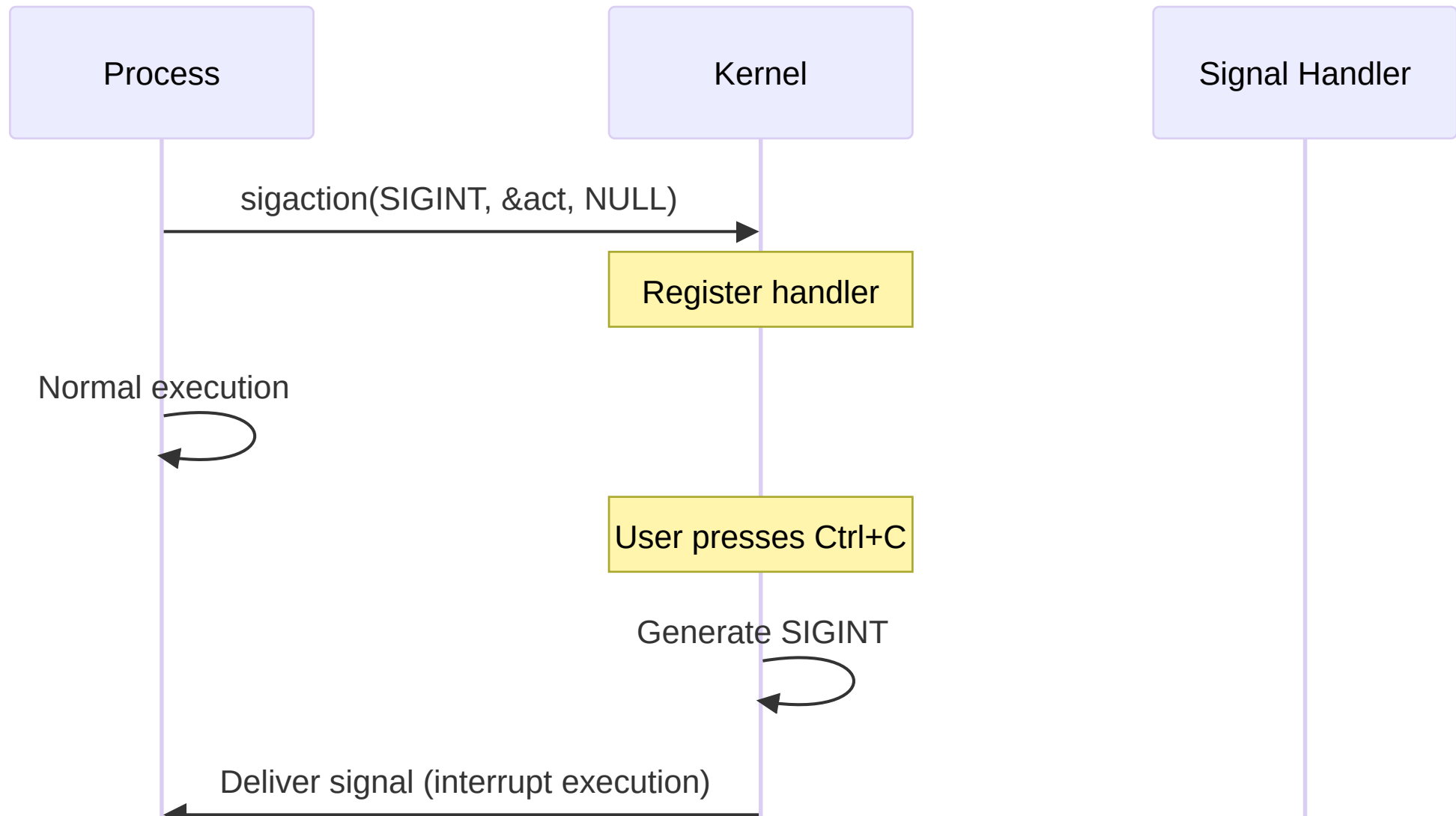
// Send signal to specific process
kill(pid, SIGTERM);    // Terminate gracefully
kill(pid, SIGKILL);    // Force kill (cannot be caught!)
kill(pid, SIGUSR1);    // User-defined signal

// Send to process group
killpg(pgid, SIGINT);

// Send to self
raise(SIGABRT);

// From shell
// kill -TERM 1234
// kill -9 1234      # SIGKILL
// killall firefox
```

Signal Handling Flow



Setting Up Signal Handlers

```
#include <signal.h>

volatile sig_atomic_t quit_flag = 0;

void signal_handler(int sig) {
    // ONLY async-signal-safe functions allowed!
    quit_flag = 1; // Set flag

    // Safe: write to stderr
    write(STDERR_FILENO, "Caught signal\n", 14);
}

int main() {
    // Simple handler (legacy, less control)
    signal(SIGINT, signal_handler);

    // Main loop
    while (!quit_flag) {
        // Do work...
    }

    printf("Exiting gracefully\n");
    return 0;
}
```

Modern Signal Handling: sigaction()

```
#include <signal.h>

void handler(int sig, siginfo_t *info, void *context) {
    // Extended handler with more information
    printf("Caught signal %d from PID %d\n",
           sig, info->si_pid);
}

int main() {
    struct sigaction sa;

    sa.sa_sigaction = handler;
    sa.sa_flags = SA_SIGINFO | SA_RESTART;
    sigemptyset(&sa.sa_mask);

    // Block SIGTERM while handling SIGINT
    sigaddset(&sa.sa_mask, SIGTERM);

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    // ... rest of program ...
}
```

Async-Signal-Safe Functions

Safe to call in signal handlers

```
// File I/O
open(), read(), write(), close()

// Process control
_exit(), fork(), wait(), waitpid()

// Signals
signal(), sigaction(), kill(), sigprocmask()

// Minimal operations
strlen() // but NOT strcpy!
```

NOT safe (never use in handlers!)

```
printf(), fprintf(), sprintf(),
```


Blocking Signals

```
#include <signal.h>

sigset_t set, oldset;

// Create signal set
sigemptyset(&set);
sigaddset(&set, SIGINT);
sigaddset(&set, SIGTERM);

// Block signals
sigprocmask(SIG_BLOCK, &set, &oldset);

// Critical section - signals pending but not delivered
// ...

// Unblock signals (deliver pending)
sigprocmask(SIG_UNBLOCK, &set, NULL);

// Restore original mask
sigprocmask(SIG_SETMASK, &oldset, NULL);
```

Real-Time Signals

```
#include <signal.h>

// Real-time signals: SIGRTMIN to SIGRTMAX
// - Queued (not lost)
// - Can carry integer payload
// - Delivered in order

// Sender
union sigval value;
value.sival_int = 42;
sigqueue(pid, SIGRTMIN, value);

// Receiver
void rt_handler(int sig, siginfo_t *info, void *context) {
    printf("RT signal %d, value=%d, from PID %d\n",
           sig, info->si_value.sival_int, info->si_pid);
}

struct sigaction sa;
sa.sa_sigaction = rt_handler;
sa.sa_flags = SA_SIGINFO;
sigemptyset(&sa.sa_mask);
sigaction(SIGRTMIN, &sa, NULL);
```

Timers and Alarms

```
#include <signal.h>
#include <unistd.h>

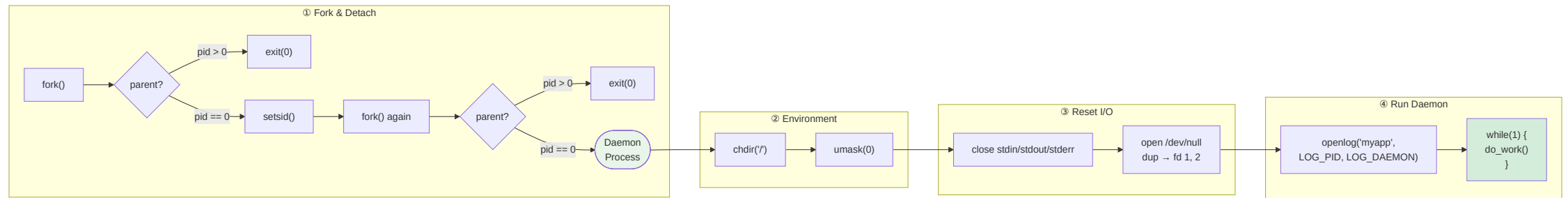
// Simple alarm (SIGALRM after N seconds)
alarm(5); // Trigger SIGALRM in 5 seconds

// Interval timer
struct itimerval timer;
timer.it_value.tv_sec = 2; // Initial delay
timer.it_value.tv_usec = 0;
timer.it_interval.tv_sec = 1; // Repeat every 1 second
timer.it_interval.tv_usec = 0;

setitimer(ITIMER_REAL, &timer, NULL); // SIGALRM
setitimer(ITIMER_VIRTUAL, &timer, NULL); // SIGVTALRM (user time)
setitimer(ITIMER_PROF, &timer, NULL); // SIGPROF (user+kernel time)
```

Section 4: Advanced Topics

Daemon Processes



Creating a Daemon

```
#include <unistd.h>
#include <sys/stat.h>
#include <syslog.h>

void daemonize() {
    // Fork and exit parent
    pid_t pid = fork();
    if (pid < 0) exit(1);
    if (pid > 0) exit(0); // Parent exits

    // Create new session
    if (setsid() < 0) exit(1);

    // Fork again to prevent TTY reacquisition
    pid = fork();
    if (pid < 0) exit(1);
    if (pid > 0) exit(0);

    // Change working directory
    chdir("/");

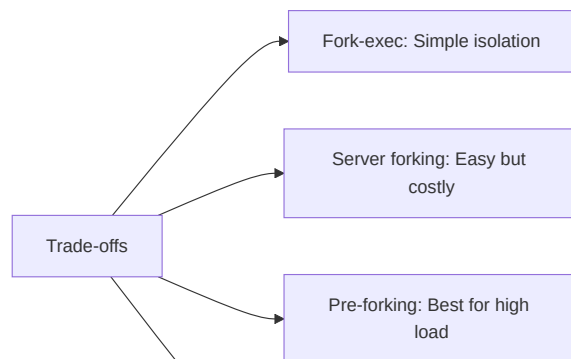
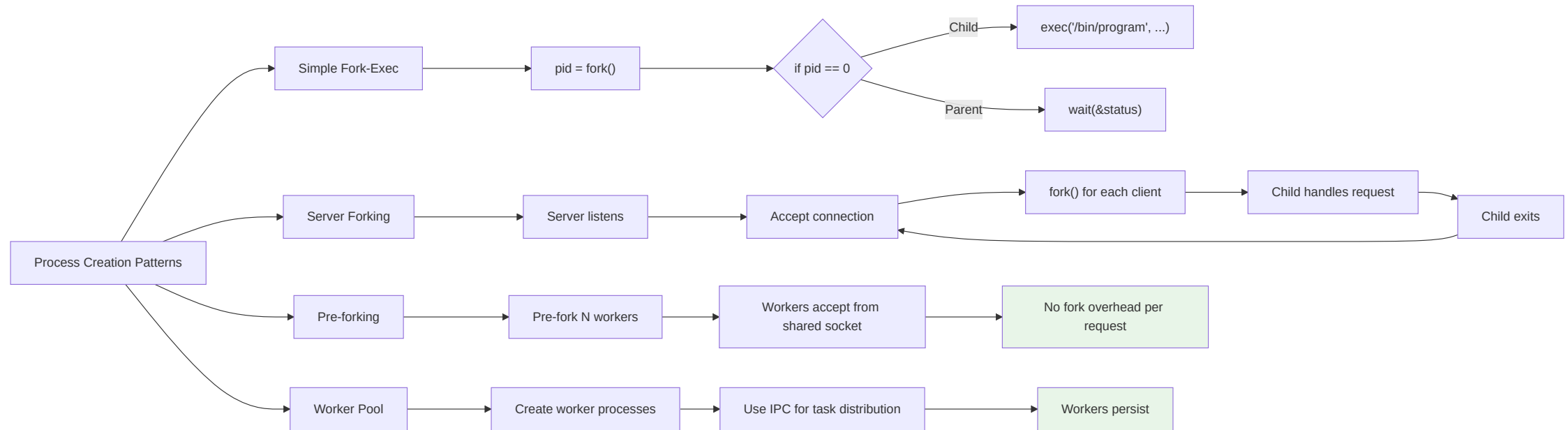
    // Reset file mode mask
    umask(0);

    // Close all file descriptors
    for (int fd = sysconf(_SC_OPEN_MAX); fd >= 0; fd--) {
        close(fd);
    }

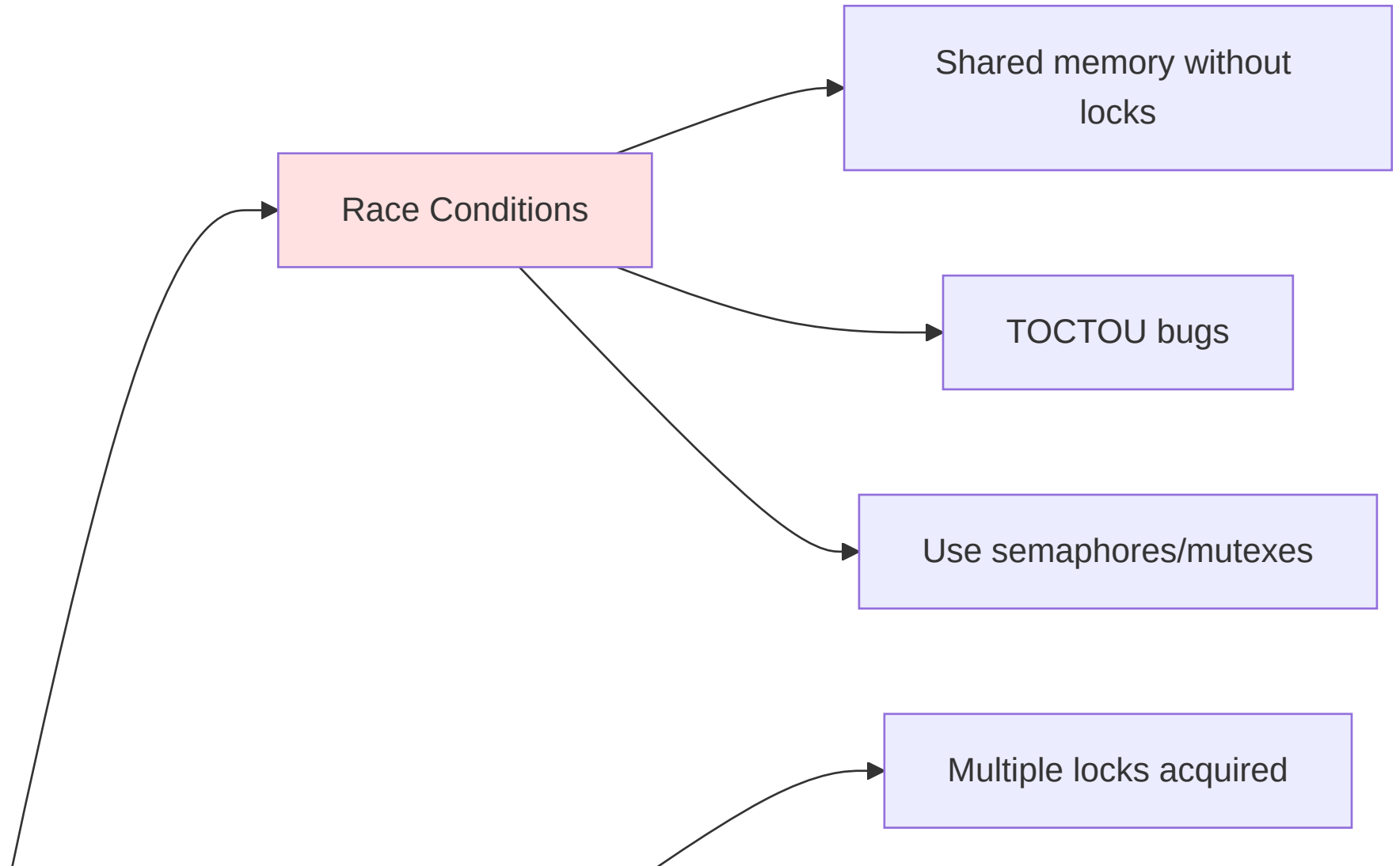
    // Redirect stdin/stdout/stderr to /dev/null
    int fd = open("/dev/null", O_RDWR);
    dup(fd);
    dup(fd);

    // Set up logging
    openlog("mydaemon", LOG_PID, LOG_DAEMON);
    syslog(LOG_INFO, "Daemon started");
}
```

Process Creation Patterns



Common IPC Pitfalls



Day 4 Summary

What We Covered

- ✓ **Processes** - fork(), exec(), wait(), lifecycle, zombies
- ✓ **IPC mechanisms** - Pipes, FIFOs, shared memory, semaphores, message queues
- ✓ **Signals** - Handlers, async-signal-safety, real-time signals
- ✓ **Daemons** - Background service creation
- ✓ **Process patterns** - Forking servers, worker pools

Key Takeaways

Critical Concepts

1. **Always wait()** for children - avoid zombies
2. **Check fork() return value** - parent gets PID, child gets 0
3. **Close unused pipe ends** - prevent deadlocks
4. **Use sigaction(), not signal()** - better control
5. **Only async-signal-safe functions** in signal handlers
6. **Shared memory needs synchronization** - use semaphores

IPC is hard - test thoroughly!

Common Mistakes

- ✗ Not handling `fork()` errors
- ✗ Forgetting to reap zombie processes
- ✗ Using `printf()` in signal handlers
- ✗ Not closing unused pipe file descriptors
- ✗ Race conditions in shared memory
- ✗ Deadlocks from incorrect semaphore usage
- ✗ Not checking IPC resource limits (`ipcs` , `ulimit`)

Debugging IPC issues is challenging - use `strace` and `gdb attach`

Hands-On Exercises

Lab Time: ~4 Hours

Navigate to: `exercises/04-fork-exec-pipes-signals.md`

You will:

1. Create processes with `fork()` and `exec()`
2. Build command pipelines with pipes
3. Implement shared memory producer-consumer
4. Handle signals gracefully
5. Create a daemon process
6. Build multi-process server architecture

Tomorrow: Day 5

Threading, Networking & Sockets

- POSIX threads (pthreads)
- Thread synchronization (mutexes, condition variables)
- Socket programming (TCP/UDP)
- Client-server architecture
- Network protocols

Exercises connect: Multi-process from Day 4 → multi-threaded Day 5

Questions?

Resources

- Process: `man 2 fork`, `man 2 exec`, `man 2 wait`
- IPC: `man 7 pipe`, `man 7 shm_overview`, `man 7 sem_overview`, `man 7 mq_overview`
- Signals: `man 7 signal`, `man 2 sigaction`, `man 7 signal-safety`
- Daemons: `man 7 daemon`

Office hours: After lab session

Additional Resources

Documentation

- [Linux Process Management](#)
- [POSIX IPC](#)
- [Signal Handling](#)
- [Unix Domain Sockets](#)

Books

- "The Linux Programming Interface" by Kerrisk (Chapters 24-34, 20-22)
- "Advanced Programming in the UNIX Environment" by Stevens (Chapters 8-10, 15)

Break!

Next Up: Hands-On Lab

 Take 10 minutes, then build multi-process applications

Remember: Processes are the foundation of Unix - master them!